```
################################################################
################################################################
--SQL CASE Statements
################################################################
################################################################


##############################
-- Task One: The SQL CASE Statement
-- In this task, we will learn how to write a conditional
-- statement using a single CASE clause
##############################

-- 1.1: Retrieve all the data in the employees table
SELECT * FROM employees;

-- 1.2: Change M to Male and F to Female in the employees table
select emp_no, first_name, last_name,
case
when gender = 'M' then 'Male'
else 'Female'
end
from employees;

-- 1.3: This gives the same result as 1.2
select emp_no, first_name, last_name,
```

```
case
when gender = 'M' then 'Male'
else 'Female'
end as gender
from employees;
-- 1.4: This gives the same result as 1.2 & 1.3
```

```
############################
-- Task Two: Adding multiple conditions to a CASE statement
-- In this task, we will learn how to add multiple conditions to a
-- CASE statement
############################

-- 2.1: Retrieve all the data in the customers table
SELECT * FROM customers;

-- 2.2: Create a column called Age_Category that returns Young for ages less than 30,
-- Aged for ages greater than 60, and Middle Aged otherwise
select*,
case
when age < 30 then 'Young'
when age > 60 then 'Aged'
else 'Middle Aged'
end as Age_Category
```

```
from customers;



-- 2.3: Retrieve a list of the number of employees that were employed before 1990, between 1990 and 1995, and

-- after 1995

SELECT * FROM employees


select emp_no, hire_date, extract(year from hire_date) as Year,

case

when extract(year from hire_date) < '1990' then 'Employed before the 90s'

when extract(year from hire_date) >= '1990' and extract(year from hire_date) <= '1995' then 'Employed between 1990 and 1095'

else 'Employed after 1995'

end as emp_date

from employees;


#############################

-- Task Three: The CASE Statement and Aggregate Functions

-- In this task, we will see how to use the CASE clause and

-- SQL aggregate functions to retrieve data

#############################


-- 3.1: Retrieve the average salary of all employees

SELECT * FROM salaries;


SELECT emp_no, AVG(salary)
```

```
FROM salaries
GROUP BY emp_no
ORDER BY AVG(salary) DESC;

-- 3.2: Retrieve a list of the average salary of employees. If the average salary is more than
-- 80000, return Paid Well. If the average salary is less than 80000, return Underpaid,
-- otherwise, return Unpaid
select emp_no, round (avg(salary), 3) as average_salary,
case
when avg(salary) > 80000 then 'Paid well'
when avg(salary) < 80000 then 'Underpaid'
else 'Unpaid'
end as Salary_category
from salaries
group by emp_no
order by average_salary desc;

-- 3.3: Retrieve a list of the average salary of employees. If the average salary is more than
-- 80000 but less than 100000, return Paid Well. If the average salary is less than 80000,
-- return Underpaid, otherwise, return Manager
select emp_no, round (avg(salary), 3) as average_salary,
case
	when avg(salary) > 80000 and avg(salary) < 10000 then 'Paid well'
	when avg(salary) < 80000 then 'Underpaid'
	else 'Manager'
end as Salary_category
```

```
from salaries
group by emp_no
order by average_salary desc;

-- 3.4: Count the number of employees in each salary category. Add a sub query
select a.salary_category, count(*)
from(
select emp_no, round (avg(salary), 3) as average_salary,
case
	when avg(salary) > 80000 and avg(salary) < 10000 then 'Paid well'
	when avg(salary) < 80000 then 'Underpaid'
	else 'Manager'
end as Salary_category
from salaries
group by emp_no
order by average_salary desc
) a
group by a.salary_category;

##############################
-- Task Four: The CASE Statement and SQL Joins
-- -- In this task, we will see how to use the CASE clause and
-- SQL Joins to retrieve data
##############################

-- 4.1: Retrieve all the data from the employees and dept_manager tables
```

```
SELECT * FROM employees
ORDER BY emp_no DESC;

SELECT * FROM dept_manager;

-- 4.2: Join all the records in the employees table to the dept_manager table
SELECT e.emp_no, dm.emp_no, e.first_name, e.last_name
FROM employees e
LEFT JOIN dept_manager dm
ON dm.emp_no = e.emp_no
ORDER BY dm.emp_no;

-- 4.3: Join all the records in the employees table to the dept_manager table
-- where the employee number is greater than 109990
SELECT e.emp_no, dm.emp_no, e.first_name, e.last_name
FROM employees e
LEFT JOIN dept_manager dm
ON dm.emp_no = e.emp_no
WHERE e.emp_no > 109990;

-- 4.4: Obtain a result set containing the employee number, first name, and last name
-- of all employees. Create a 4th column in the query, indicating whether this
-- employee is also a manager, according to the data in the
-- dept_manager table, or a regular employee
select e.emp_no, e.first_name, e.last_name,
case
```

```
        when dm.emp_no is not null then 'Manger'
        else 'Employee'
        end as is_manager
        from employees e
        left join dept_manager dm
        on dm.emp_no = e.emp_no
        order by dm.emp_no;
```

```
-- 4.5: Obtain a result set containing the employee number, first name, and last name
-- of all employees with a number greater than '109990'. Create a 4th column in the query,
-- indicating whether this employee is also a manager, according to the data in the
-- dept_manager table, or a regular employee
select e.emp_no, e.first_name, e.last_name,
case
        when dm.emp_no is not null then 'Manger'
        else 'Employee'
end as is_manager
from employees e
left join dept_manager dm
on dm.emp_no = e.emp_no
where e.emp_no > 109990
order by dm.emp_no;
```

##############################

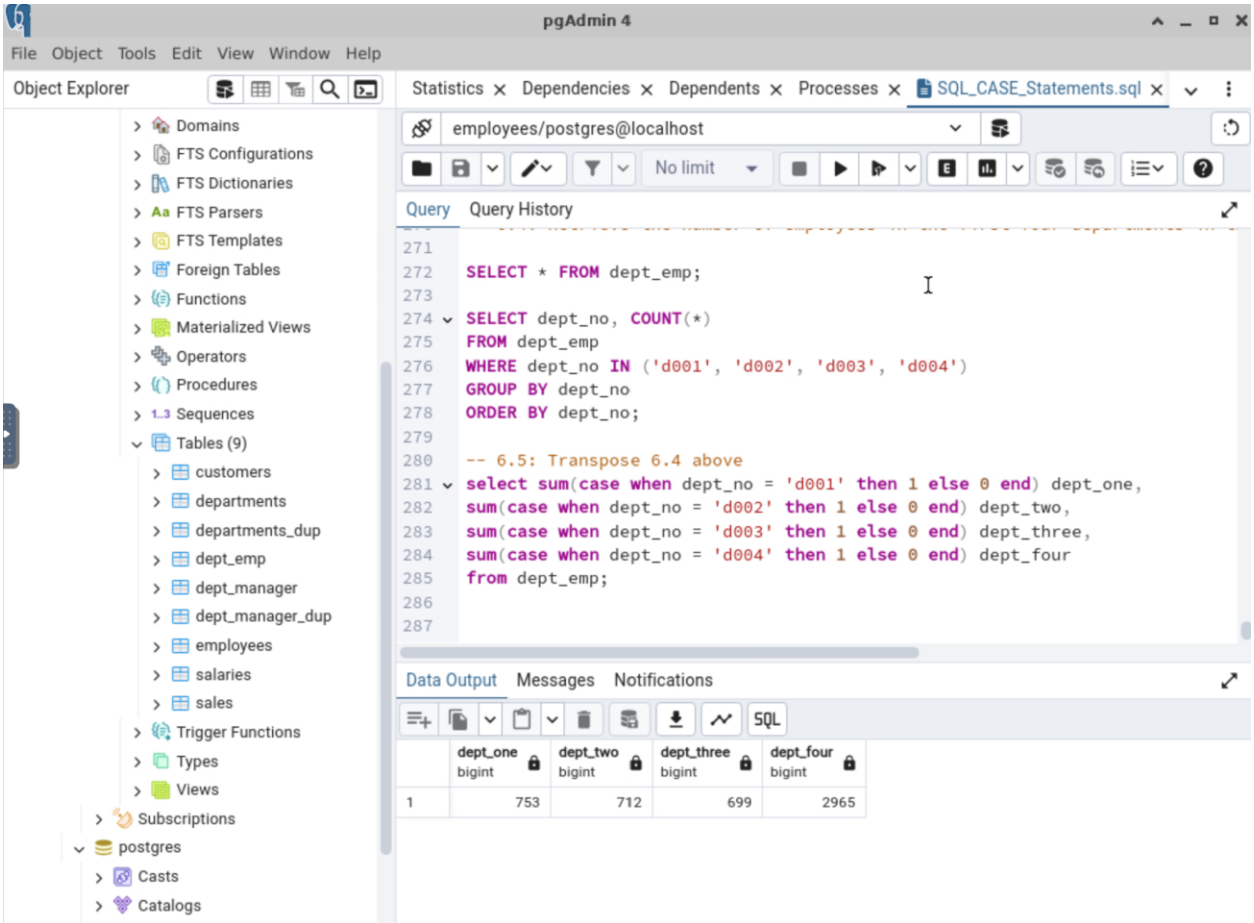
```
-- Task Five: The CASE Statement together with Aggregate Functions and Joins
-- In this task, we will see how to use the CASE clause together with
-- SQL aggregate functions and SQL Joins to retrieve data
##############################

-- 5.1: Retrieve all the data from the employees and salaries tables
SELECT * FROM employees;

SELECT * FROM salaries;

-- 5.2: Retrieve a list of all salaries earned by an employee
SELECT e.emp_no, e.first_name, e.last_name, s.salary
FROM employees e
JOIN salaries s
ON e.emp_no = s.emp_no;

/* 5.3: Retrieve a list of managers employee number, first name and last name.
Add a column called 'salary difference' which is the difference between the
manager’s maximum and minimum salary. Also, add a column called
'salary_increase', which returns 'Salary was raised by more than $30,000' if the difference
is more than $30,000, 'Salary was raised by more than $20,000 but less than $30,000',
if the difference is between $20,000 and $30,000, 'Salary was raised by less than $20,000'
if the difference is less than $20,000 */
select e.emp_no, e.first_name, e.last_name, max (s.salary)- min (s.salary)as
salary_difference,
case
```

```
        when max(s.salary)- min(s.salary)>30000 then 'SAlary was raised by more than $30,000'
        when max(s.salary)- min(s.salary) between 20000 and 30000
        then 'Salary was raised by more than $20,000 but less than $30,000'
        else 'Salary was raised by less than $20,000'
end as Salary_increse
from employees e
join salaries s
on e.emp_no= s.emp_no
group by e.emp_no, s.emp_no;

-- 5.4: Retrieve all the data from the employees and dept_emp tables
SELECT * FROM employees;

SELECT * FROM dept_emp;

/* 5.5: Extract the employee number, first and last name of the first 100 employees,
and add a fourth column called "current_employee" saying "Is still employed",
if the employee is still working in the company, or "Not an employee anymore",
if they are not more working in the company.
Hint: We will need data from both the 'employees' and 'dept_emp' table to solve this exercise */

select e.emp_no, e.first_name, e.last_name,
case
        when max (de.to_date)>current_date then 'Is still employed'
        else 'Not an employee anymore'
```

```
end as current_employee
from employees e
join dept_emp de
on e.emp_no = de.emp_no
group by e.emp_no, de.to_date
limit 100;



#############################
-- Task Six: Transposing data using the CASE clause
-- In this task, we will learn how to use the SQL CASE statement to
-- transpose retrieved data
#############################

-- 6.1: Retrieve all the data from the sales table
SELECT * FROM sales;

-- 6.2: Retrieve the count of the different profit_category from the sales table
SELECT a.profit_category, COUNT(*)
FROM (
SELECT order_line, profit,
CASE
        WHEN profit < 0 THEN 'No Profit'
        WHEN profit > 0 AND profit < 500 THEN 'Low Profit'
        WHEN profit > 500 AND profit < 1500 THEN 'Good Profit'
        ELSE 'High Profit'
```

```
END AS profit_category
FROM sales
) a
GROUP BY a.profit_category;

-- 6.3: Transpose 6.2 above (the previus colums are not in a single row)
select sum(case when profit < 0 then 1 else 0 end) as no_profit,
sum(case when profit > 0 and profit < 500 then 1 else 0 end) as low_profit,
sum(case when profit > 1500 then 1 else 0 end) as high_profit
from sales;


-- 6.4: Retrieve the number of employees in the first four departments in the dept_emp table

SELECT * FROM dept_emp;

SELECT dept_no, COUNT(*)
FROM dept_emp
WHERE dept_no IN ('d001', 'd002', 'd003', 'd004')
GROUP BY dept_no
ORDER BY dept_no;

-- 6.5: Transpose 6.4 above
select sum(case when dept_no = 'd001' then 1 else 0 end) dept_one,
sum(case when dept_no = 'd002' then 1 else 0 end) dept_two,
```

sum(case when dept_no = 'd003' then 1 else 0 end) dept_three,

sum(case when dept_no = 'd004' then 1 else 0 end) dept_four

from dept_emp;